

CS787: GENERATIVE ARTIFICIAL INTELLIGENCE TRANSFORMERS

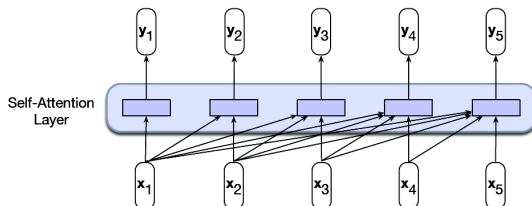
Arnab Bhattacharya
arnabb@cse.iitk.ac.in

Computer Science and Engineering,
Indian Institute of Technology, Kanpur
<http://web.cse.iitk.ac.in/~cs787/>

1st semester, 2026-27

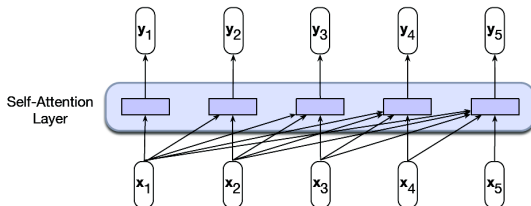
Wed 12:00–13:15 at L20, Fri 12:00–13:15 at L19

Transformers



- **Transformers** are *self-attention networks*
- Each unit i uses a *weighted* version of previous units j as additional input
- This is called **attention** (or **self-attention**)

Transformers



- **Transformers** are *self-attention networks*
- Each unit i uses a *weighted* version of previous units j as additional input
- This is called **attention** (or **self-attention**)
- Weight depends on similarity between these two units

$$\mathbf{x}_i = \sum_{\forall j \leq i} \alpha_{ij} \cdot \mathbf{x}_j$$

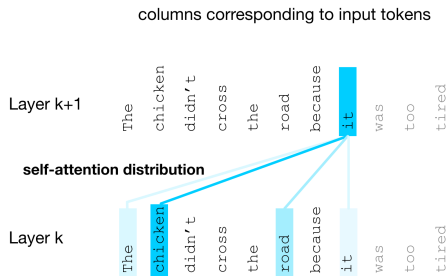
$$\alpha_{ij} = \text{softmax}(\mathbf{x}_i \cdot \mathbf{x}_j) = \frac{\exp(\mathbf{x}_i \cdot \mathbf{x}_j)}{\sum_{\forall j} \exp(\mathbf{x}_i \cdot \mathbf{x}_j)} \quad \forall j \leq i$$

Attention Intuition

- Two sentences
 - The chicken didn't cross the road because *it* was too tired
 - The chicken didn't cross the road because *it* was too wide

Attention Intuition

- Two sentences
 - The chicken didn't cross the road because *it* was too tired
 - The chicken didn't cross the road because *it* was too wide
- Contextual embedding of the word “it”
 - Word2Vec, etc. produces static embedding



Attention Details

- Attention output is sum of weighted contexts:

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \text{ where } \alpha_{ij} = \textit{softmax}(\textit{score}(\mathbf{x}_i, \mathbf{x}_j))$$

- Score or similarity between two words is

$$\textit{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

Attention Details

- Attention output is sum of weighted contexts:

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \text{ where } \alpha_{ij} = \textit{softmax}(\textit{score}(\mathbf{x}_i, \mathbf{x}_j))$$

- Score or similarity between two words is

$$\textit{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

- Attention of $\mathbf{x}_i$ to itself is high
- A word plays different roles when it attends to versus when it is attended to

Query, Key and Value

- Each input can be used in three different ways

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \cdot \mathbf{x}_j = \sum_{j \leq i} f(\mathbf{x}_i, \mathbf{x}_j) \cdot \mathbf{x}_j$$

- **Query** $\mathbf{q}_i$: As embedding at current time t
- **Key** $\mathbf{k}_i$: As embedding at some previous time
- **Value** $\mathbf{v}_i$: Output of current time t

Query, Key and Value

- Each input can be used in three different ways

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \cdot \mathbf{x}_j = \sum_{j \leq i} f(\mathbf{x}_i, \mathbf{x}_j) \cdot \mathbf{x}_j$$

- Query** $\mathbf{q}_i$: As embedding at current time t
- Key** $\mathbf{k}_i$: As embedding at some previous time
- Value** $\mathbf{v}_i$: Output of current time t
- Implemented as weight matrices

$$\mathbf{q}_i = \mathbf{x}_i W^Q; \quad \mathbf{k}_i = \mathbf{x}_i W^K; \quad \mathbf{v}_i = \mathbf{x}_i W^V;$$

- Score of $\mathbf{x}_i$ is, therefore,

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$
$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))$$

Single-Head Attention

- Output vector for unit i

$$\begin{aligned}\mathbf{head}_i &= \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j \\ &= \sum_{j \leq i} \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \right) \mathbf{v}_j\end{aligned}$$

Single-Head Attention

- Output vector for unit i

$$\begin{aligned}\mathbf{head}_i &= \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j \\ &= \sum_{j \leq i} \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \right) \mathbf{v}_j\end{aligned}$$

- **Single-attention head** $\mathbf{head}_i$
- W^O reshapes $\mathbf{a}_i$ to have dimensionality d from d_v for final output $\mathbf{a}_i$

$$\mathbf{a}_i = \mathbf{head}_i \cdot W^O = \left(\sum_{j \leq i} \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \right) \mathbf{v}_j \right) W^O$$

Single-Head Attention

- Output vector for unit i

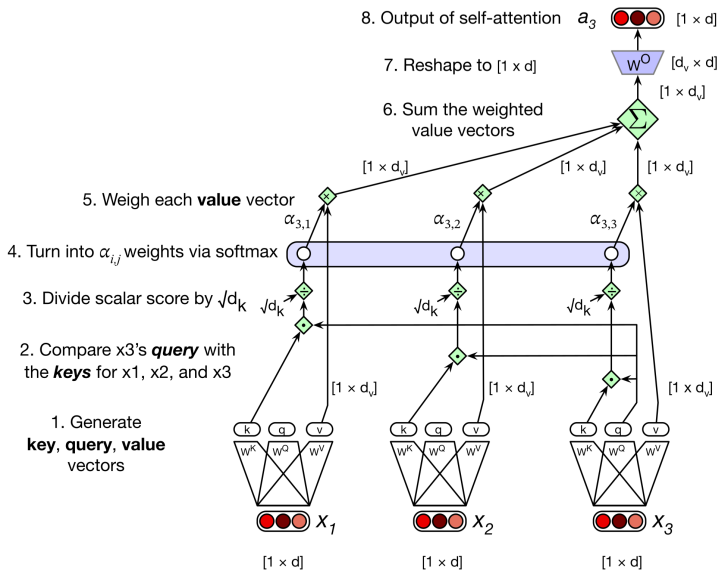
$$\begin{aligned}\mathbf{head}_i &= \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j \\ &= \sum_{j \leq i} \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \right) \mathbf{v}_j\end{aligned}$$

- **Single-attention head** $\mathbf{head}_i$
- W^O reshapes $\mathbf{a}_i$ to have dimensionality d from d_v for final output $\mathbf{a}_i$

$$\mathbf{a}_i = \mathbf{head}_i \cdot W^O = \left(\sum_{j \leq i} \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \right) \mathbf{v}_j \right) W^O$$

- Dimensionalities
 - d_k for query and key
 - d_v for value
 - d for final output

Schematic



Multi-Head Attention

- Attention can be **multi-head**
- To attend to different properties of the previous word

Multi-Head Attention

- Attention can be **multi-head**
- To attend to different properties of the previous word
- For each head c , there are separate W^{Q_c} , W^{K_c} , W^{V_c} matrices
- For t attention heads

$$\mathbf{q}_i^c = \mathbf{x}_i \cdot W^{Q_c} \quad \mathbf{k}_i^c = \mathbf{x}_i \cdot W^{K_c} \quad \mathbf{v}_i^c = \mathbf{x}_i \cdot W^{V_c}$$

$$score^c(\mathbf{x}_i, \mathbf{x}_j) = softmax\left(\frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k^c}}\right) \mathbf{v}_j^c \quad \forall j \leq i$$

$$\mathbf{head}_i^c = \sum_{j \leq i} score^c(\mathbf{x}_i, \mathbf{x}_j) \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\mathbf{head}_i^1 \oplus \mathbf{head}_i^2 \oplus \dots \oplus \mathbf{head}_i^t) \cdot W^O$$

where $\oplus$ is *concatenation*

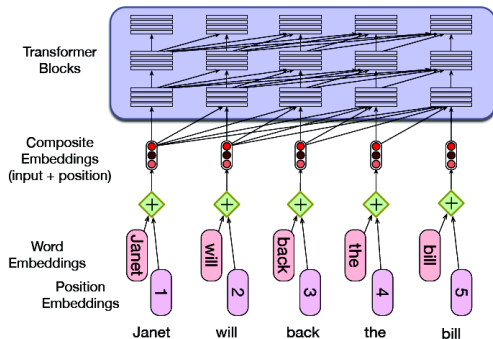
- Reshaping by W^O is necessary

Sequence of Words

- So far, previous time steps act like a bag of words

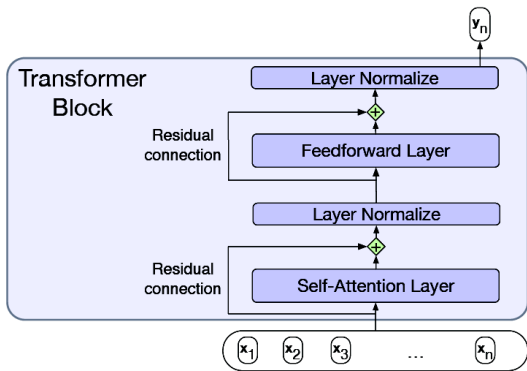
Sequence of Words

- So far, previous time steps act like a bag of words



- To encode sequence, **position vectors** (**position embeddings**) are used
- Embeddings for *absolute* positions are also learnt
 - May be static or sinusoidal
- Can be *relative* positions as well
 - *Rotary Positional Encoding (RoPE)* is common

Transformer Block



- Stacked layers form a **transformer block**
- *Residual connections* short-circuit information by bypassing a layer
- *Layer normalization* converts to z-scores
 - Pre-norm stabilises gradients more
 - Post-norm creates superior features
- *Feedforward layer* is fully connected with 1 hidden layer, and ReLU

Transformer Block Layers

- Input is $\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i$

$$\mathbf{t}_i^{(1)} = \text{LayerNorm}(\mathbf{x}_i) \text{ (using } \mu, \sigma \text{)}$$

$$\mathbf{t}_i^{(2)} = \text{MultiHeadAttention}(\mathbf{t}_i^{(1)}) \text{ (token mixing)}$$

$$\mathbf{t}_i^{(3)} = \mathbf{t}_i^{(2)} + \mathbf{x}_i \text{ (residual connections)}$$

$$\mathbf{t}_i^{(4)} = \text{LayerNorm}(\mathbf{t}_i^{(3)}) \text{ (using } \mu, \sigma \text{)}$$

$$\mathbf{t}_i^{(5)} = \text{FeedForward}(\mathbf{t}_i^{(4)}) \text{ (2-layer ANN)}$$

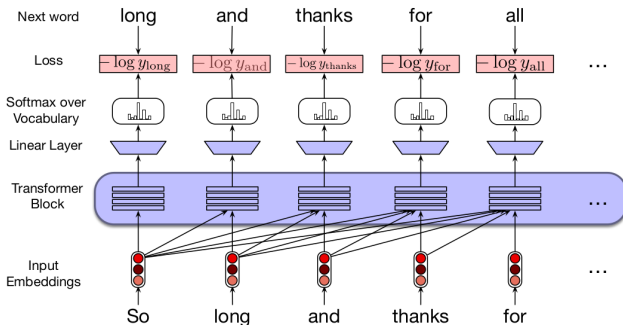
$$\mathbf{h}_{i_1} = \mathbf{t}_i^{(5)} + \mathbf{t}_i^{(3)} \text{ (residual connections)}$$

$$\mathbf{x}_{i_2} = \mathbf{h}_{i_1} \text{ (next block continues)}$$

...

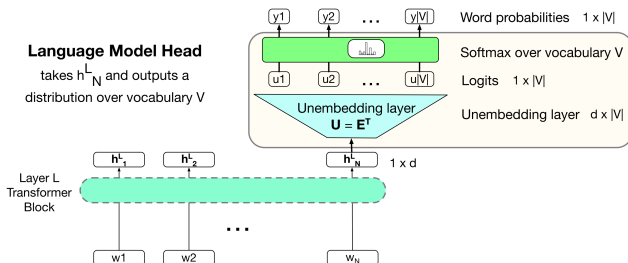
- Final output is $\mathbf{h}_{i_L}$ after L transformer blocks

Language Modeling using Transformers



- RNN is replaced by transformer block
- Last layer on top of transformer block predicts next word

Language Modeling Head



- Language modeling layer on top of last transformer block
- *Unembedding layer* expands from dimensionality d to V
- Can be simply E^T (transpose of embedding vector)
- Softmax to predict word

Next Word

- Which word to predict next?

Next Word

- Which word to predict next?
- Highest probably word: accurate but repetitive (and boring)

Next Word

- Which word to predict next?
- Highest probably word: accurate but repetitive (and boring)
- Sample from top- k words
 - Renormalize first k words
 - k is fixed

Next Word

- Which word to predict next?
- Highest probably word: accurate but repetitive (and boring)
- Sample from top- k words
 - Renormalize first k words
 - k is fixed
- Nucleus or top- p sampling
 - Pick words that add up to p probability mass